Review notes · for you

Build an agent in Python: unit check

AI agents · Upper secondary · Unit check

Time	Materials
20 min	Paper and a pen. No device needed.

Learning goals

- Read a tool call
- Tools are functions
- The agent loop
- Guardrails in code

Activity: Build the agent on paper

1. Warm-up (3 min): Without notes, write the four parts: the parser, the tools, the loop and the guard.
2. Draw (7 min): Draw the whole agent: the model's text going in, the parser, the guard, a person, the tools and the history, with arrows.
3. Break it (7 min): Swap drawings. Your partner invents a model output that gets through, such as a broken call, a made-up tool, a model that never answers or a message nobody approved.
4. Fix (3 min): Add the rule, or the test, that would have stopped it.

Use made-up tools, places and people only.

Discussion

- Which part of your agent was hardest to get right?
- What could a real model do that your scripted model never did?
- Where would you put a person in the loop, and why?

Check yourself

- One guardrail I would add to every agent, and why.

Try it this week

Pick one part of your agent (the parser, the tools, the loop or the guard) and add one test that would have caught a real mistake from this unit, such as 9Z copied as 9C or a guard checking the wrong tool name.

Quiz sheet

Build an agent in Python: unit check

AI agents · Upper secondary · Unit check

Name _____ Class _____ Date _____

1 **On each turn, what does an agent's model actually produce?**

◯ The tool's result ◯ A finished action, such as a sent message

◯ Text: a plan, a tool call or an answer

2 **json.loads turns the JSON text of a call's details into a Python dictionary.**

◯ True ◯ False

3 **Put the lines of run in order.**

Number the lines in order (1, 2, 3…), and draw one arrow → in front of a line for each indent it needs. Cross out any line that doesn't belong.

☐ `if name not in TOOLS:`

☐ `def run(name, args):`

☐ `return TOOLS[name](args)`

☐ `return TOOLS[args](name)`

☐ `return {"error": "unknown tool"}`

4 **What does this program print?**

```
import json
text = 'weather{"place":"Kafi"'
try:
    json.loads(text[text.find("{"):])
    print("call")
except ValueError:
    print("broken call")
```

◯ Nothing: an error stops the program ◯ call ◯ broken call

5 **Match each part of your agent to what it protects against.**

try / except ValueError	• • • •	A made-up tool causing a KeyError
if name not in TOOLS	• • • •	A message nobody approved
for step in range(limit)	• • • •	A broken call crashing the agent
approved != args	• • • •	A model that never answers

6 **This parse_call fails on every call. Which line is the mistake?**

Circle the line with the mistake. Write it correctly:

```
1  def parse_call(text):
2      i = text.find("{")
3      if i == -1:
4          return None
5      args = json.loads(text[i + 1:])
6      return text[:i], args
```

---

7 **A guard allows weather and send_message. Which of these should it NOT simply run? Choose all that apply.**

- ☐ weather for Kafi
- ☐ send_message, with a yes to this exact message
- ☐ send_message, with a yes to a different message
- ☐ A call to attendance
- ☐ send_message to Mrs Mensah, with no yes yet

8 **Put in order what your agent does with one tool call.**

- ☐ Log the decision and who approved it, before running it
- ☐ Split the call into a name and details
- ☐ Add the result to the history
- ☐ Run the tool
- ☐ If the guard says “ask”, ask the person about this exact action
- ☐ Check it with the guard

9 **Fill the gaps so the loop can never run for ever.**

```
def agent(request, limit=4):
    history = [request]
    for step in __________(limit):
        out = model(history)
        if "{" not in out:
            __________ out
        history += [out, tool(out)]
    return "Stopped"
```

Word bank: `return` `while` `len` `range` `print`

10 **In this course’s code steps the model is scripted, so passing the tests shows that your code works, not that a real model will behave.**

- ◯ True
- ◯ False

Check your answers

Build an agent in Python: unit check

AI agents · Upper secondary · Unit check

1 **Text: a plan, a tool call or an answer**

The model only ever writes text. The tool's result comes from your code running the tool, and any action happens only if your code runs it.

2 **True**

That is its whole job: text in, a dictionary out.

3 **Top to bottom: 2, 1, 4, X, 3**

```
def run(name, args):
    if name not in TOOLS:
        return {"error": "unknown tool"}
    return TOOLS[name](args)
```

Check the name first and return an error for an unknown tool; then TOOLS[name] is the function, called with the details.

4 **broken call**

The JSON is missing its closing bracket, so json.loads raises a ValueError; except catches it and the program prints "broken call" instead of crashing.

5 **try / except ValueError — A broken call crashing the agent**
if name not in TOOLS — A made-up tool causing a KeyError
for step in range(limit) — A model that never answers
approved != args — A message nobody approved

Each piece of code guards against one way a model's text can go wrong: broken JSON, a tool that doesn't exist, a loop that never ends, and an action nobody approved.

6 **Line 5: `args = json.loads(text[i:])`**

text[i + 1:] drops the opening bracket, so the JSON is broken and json.loads raises an error.

7 **A call to attendance; send_message to Mrs Mensah, with no yes yet; send_message, with a yes to a different message**

attendance isn't on the allow-list, so it's blocked.

8 **Top to bottom: 4, 1, 6, 5, 3, 2**

You can only check a call once it's split; the guard decides whether a person must approve this exact action; the decision and who approved it are logged before anything runs; and only then does the tool run and its result join the history.

9

```
def agent(request, limit=4):
    history = [request]
    for step in range(limit):
        out = model(history)
        if "{" not in out:
            return out
        history += [out, tool(out)]
    return "Stopped"
```

range(limit) caps the number of turns, an answer is returned at once, and after the last step the loop returns "Stopped".

10 **True**

A script returns exactly what you wrote.